

# Codex Web Relay (魔法玻璃旗艦版) 部署指南

本指南將協助您將 **Codex Web Relay** 專案透過 Docker 分別部署至 **Windows (Docker Desktop)** 與 **Linux (VPS)** 環境。Docker 部署能有效隔離環境，減少相依性衝突，是生產環境的最佳實踐。

## 一、專案準備 (通用步驟)

無論您選擇哪種系統，請先在主機上建立一個專屬資料夾（例如 `codex-relay`），並確保資料夾內**精確包含**以下三個檔案：

### 1. 核心 Python 腳本 (`codex_web_relay.py`)

#### ⚠ Windows 用戶致命陷阱：

Windows 預設會「隱藏已知檔案類型的副檔名」。如果您自己新增檔案並命名為 `codex_web_relay.py`，它實際上可能變成了 `codex_web_relay.py.py`。

**解決方案：**打開資料夾的「檢視」> 勾選「副檔名」，確保檔名精確無誤，否則 Docker 啟動時會報錯 `[Errno 2] No such file or directory`。

### 2. 建立 Dockerfile

在同目錄下新建無副檔名的 `Dockerfile`，內容如下：

```
# 使用輕量級 Python 鏡像
FROM python:3.11-slim

# 設定工作目錄
WORKDIR /app

# 複製當前目錄下的所有檔案到容器
COPY . .

# 安裝必要的依賴
RUN pip install --no-cache-dir fastapi uvicorn httpx pydantic

# 開放腳本中定義的 4446 端口
EXPOSE 4446

# 啟動命令
CMD ["python", "codex_web_relay.py"]
```

---

## 二、Windows 環境部署指南 (Docker Desktop)

在 Windows 下，Docker Desktop 實際上運行在一個隱藏的虛擬機中，因此無法使用原生的 Host 網路模式。我們必須使用「端口映射」，並利用魔法地址來連線本機服務。

## 1. 建立 docker-compose.yml (Windows 專用)

```
version: '3.8'

services:
  codex-relay:
    build: .
    container_name: codex_relay
    restart: always
    ports:
      # 映射端口：讓你的 Windows 瀏覽器能訪問容器
      - "4446:4446"
    volumes:
      - /etc/localtime:/etc/localtime:ro
    environment:
      - PYTHONUNBUFFERED=1
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

## 2. 啟動與連線設定

- 在專案資料夾開啟 PowerShell，執行：`docker-compose build --no-cache`，然後執行 `docker-compose up -d`。
- 瀏覽器開啟網頁控制台：`http://127.0.0.1:4446`。

### 連線 Windows 本機的 Ollama：

由於容器網路被隔離，若要在網頁端連線 Windows 主機上的 Ollama，請在網頁配置中心的 Base URL 填寫 Docker 魔法地址：

`http://host.docker.internal:11434/v1`

## 三、Linux 環境部署指南 (VPS)

在 Linux 環境下，Docker 原生支援 `host` 網路模式，這能讓容器與主機完美共享網路棧，不需複雜的地址穿透，效能與設定最為直接。

## 1. 建立 docker-compose.yml (Linux 專用)

```
version: '3.8'

services:
  codex-relay:
    build: .
    container_name: codex_relay
    restart: always
    network_mode: "host" # Linux 原生支援，直接共享主機網路
    volumes:
      - /etc/localtime:/etc/localtime:ro
    environment:
      - PYTHONUNBUFFERED=1
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

## 2. 啟動與連線設定

- 上傳專案至 VPS，在終端機進入目錄後執行：`docker-compose build --no-cache` 及 `docker-compose up -d`。
- 瀏覽器開啟網頁控制台：`http://你的VPS公網IP:4446`。

### 💡 連線 Linux 本機的 Ollama：

因為啟用了 `host` 模式，容器等於主機本身。若 Ollama 安裝在同一台 VPS，網頁配置中心的 Base URL 請直接填寫：

`http://127.0.0.1:11434/v1`

---

## 四、Nginx 反向代理與最佳實踐 (重要)

強烈建議不要在生產環境的公網直接暴露 4446 端口。您應該使用 Nginx (或 Nginx Proxy Manager) 綁定域名並加上 HTTPS (SSL) 憑證。

### ⚠️ 支援打字機效果 (Server-Sent Events) 的 Nginx 關鍵配置：

為了讓 AI 的回覆能夠流式輸出（逐字顯示而不卡頓），您必須在 Nginx 代理設定中關閉緩衝：

```
location / {
    proxy_pass http://127.0.0.1:4446;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    # 解決 SSE (打字機效果) 延遲與卡頓的關鍵
    proxy_buffering off;
    proxy_cache off;
    chunked_transfer_encoding off;
}
```

## 五、常見問題排查 (Troubleshooting)

- **網頁空白**：通常是 Python 腳本更新時字串跳脫錯誤導致，請確保使用最新的 raw string 原始字串版本腳本。
- **日誌出現 [Errno 2]**：Dockerfile 找不到腳本。檢查 Linux/Windows 下檔名是否被加上了隱藏的 `.py`。重命名後必須執行 `docker-compose build --no-cache`。
- **API 報錯 (Context Length Exceeded)**：Codex App 對話歷史過長，超過模型上限。請在客戶端一鍵「清空歷史 (Clear History)」即可解決。
- **API 報錯 (User Not Found/401)**：API Key 填寫錯誤或失效。請在配置中心點擊「顯示完整」檢查，或重新生成 Key。